

msudoku

MATLAB app that provides a Sudoku GUI with puzzle generation, solving, and validation helpers.

Project Requirements Checklist

Required Conditions

- **Game board (playing area) stored in a matrix**
 - The 9x9 Sudoku grid is stored in matrices (`puzzle`, `fullGrid`) and displayed using `uicontrol` edit boxes.
- **Use exactly 2–3 commands that are not mentioned in any PDF**
 - `uigetfile` - open file selection dialog box
 - `feof` - test for end of file
 - `randperm` - random permutation of integers
- **Functions, for-loops, while, if, switch used in the form shown in the exercises**
- **Working with matrices and vectors as practiced in the exercises**
- **Use 3 levels/difficulty tiers**
 - Three difficulty levels: Easy (30 blanks), Medium (40 blanks), Hard (50 blanks).
- **Custom map in .txt, at least 2 versions**
 - Two custom puzzle files: `puzzle1.txt` and `puzzle2.txt`
 - Load functionality via the “Load Puzzle” button or input parameter.

Code

- **At the beginning: briefly introduce the game – rules, controls**
 - Game introduction displayed via `msgbox` when `sudoku_gui()` is called (can be disabled with `sudoku_gui(false)`).
 - Introduction includes: rules, controls, and scoring system.
- **Any input parameters**
 - Function: `sudoku_gui(showIntro, loadPuzzleFile)`
 - `showIntro`: logical, controls whether to show an introduction (default: `true`)
 - `loadPuzzleFile`: string, path to .txt file to load puzzle from (default: `''`)

Example Usage with Parameters:

```
% Start with introduction  
sudoku_gui()
```

```
% Start without introduction
```

```
sudoku_gui(false)
```

```
% Start and load a puzzle file
```

```
sudoku_gui(true, 'puzzle1.txt')
```

```
% Start without introduction and load puzzle
```

```
sudoku_gui(false, 'puzzle2.txt')
```

- **Winner and number of moves/achieved score**

- Winner detection: When the puzzle is completed correctly, displays “WIN! Completed in X moves”
- Move counter: Tracks and displays the number of moves
- Score tracking: Shows mistakes count (0/5) and the game ends after five mistakes
- The status bar shows the current game state and move count

Aesthetics

- **Error resistance**

- Input validation: Only accepts integers 1-9
- File loading: Validates file existence, format (9x9), and value ranges (0-9)
- Error handling: Try-catch blocks, null checks, handle validation
- Invalid input feedback: Status messages guide user

- **GUI – Programmatic Creation (Octave Compatible)**

- Instead of a static .fig file, the GUI is generated dynamically using the `figure` command.
- **Handle Management:** Uses `gcf` (Get Current Figure) to identify the active window during callbacks.
- **Data Persistence:** Uses `setappdata` and `getappdata` to share the Sudoku matrix and move counters between separate .m files.
- **Uicontrols:** All 81 grid cells and game buttons are created as handles, allowing for real-time color and text updates.

Creativity

1. **Custom Puzzle Loading** - Load puzzles from .txt files with file dialog
2. **Mistake Limit System** - Game ends after five mistakes with visual feedback
3. **Conflict Highlighting** - Visual highlighting of conflicts in red
4. **Move Counter** - Real-time tracking and display of moves
5. **Clear Entries vs Clear All** - Two-level clearing system
6. **Auto-solver** - Backtracking algorithm to solve puzzles
7. **Status Bar** - Comprehensive status messages throughout gameplay

Game Rules

- Fill the 9x9 grid with digits 1-9
- Each row must contain all digits 1-9 exactly once
- Each column must contain all digits 1-9 exactly once
- Each 3x3 block must contain all digits 1-9 exactly once

Controls / How to Play

1. **Start the game:** Run `sudoku_gui` in MATLAB
2. **Generate puzzle:** Click “New (Easy/Medium/Hard)” to create a new puzzle
3. **Enter numbers:** Click on cells and type digits 1-9
4. **Check:** Click “Check” to validate entries and find conflicts (highlighted in red)
5. **Solve:** Click “Solve” to auto-complete the puzzle
6. **Clear Entries:** Remove only your entered digits (keeps original puzzle)
7. **Clear All:** Reset the entire board
8. **Load Puzzle:** Load a custom puzzle from a .txt file
9. **Close:** Close the window using the standard window close button

File Overview

Main Files

- `sudoku_gui.m`: UI creation and shared state initialization (with input parameters and introduction)
- `generate_button_callback.m`: puzzle generation for selected difficulty
- `cell_edit_callback.m`: per-cell validation and move counter updates
- `check_button_callback.m`: grid validation and highlighting
- `solve_button_callback.m`: solver entry point and UI updates
- `clear_entries_callback.m`: wipes only user-entered digits
- `clear_button_callback.m`: complete board reset
- `load_puzzle_callback.m`: file dialog for loading puzzles
- `load_puzzle_from_file.m`: loads puzzle from .txt file

Game Logic

- `generate_full.m`: random full solution generator.
- `make_puzzle.m`: removes clues from a full grid.
- `solve_iterative.m`: iterative backtracking solver.
- `can_place.m`: Sudoku rule helper for row/column/box validation.

GUI Utilities

- `read_grid_from_handles.m`: extracts grid data using linear indexing.
- `highlight_cells.m`: colors cell handles (white for normal, red for error).

- `update_status.m`: updates the GUI status bar text.
- `set_buttons_enabled.m`: toggles Enable state for ‘Check’ and ‘Clear Entries’ buttons.
- `apply_theme`: updates UI to match the selected visual style.

Custom Puzzle Files

- `puzzle1.txt`: Custom Sudoku puzzle #1
- `puzzle2.txt`: Custom Sudoku puzzle #2